

SENIOR SWE INTERVIEW SERIES

Module 5: DNS

Interview-focused networking, as asked at Google, Meta, Amazon, Microsoft, Uber, Netflix, Cloudflare, Stripe, LinkedIn & Cisco

Part of: Computer Networking for Senior Software Engineer Interviews

Built July 02, 2026

MODULE 5 — DNS

DNS appears in every “what happens when you type a URL” question, powers CDN/GSLB routing, and causes a disproportionate share of famous outages (“It’s always DNS”). Interviewers use it to test both fundamentals and operational judgment.

Topic 5.1 — DNS Resolution (End-to-End)

1. Why Interviewers Ask This

“Walk me through what happens when you type example.com” starts here. Senior candidates are expected to know the cache hierarchy, who talks to whom, and modern realities (resolver anycast, DoH, negative caching).

2. Core Concept

DNS maps names → records (A/AAAA = IP, CNAME = alias, MX, TXT, NS, SRV). Resolution walks a delegation hierarchy: **root** → **TLD** (.com) → **authoritative** (example.com’s servers), but caching at every layer means the full walk is rare (<1% of queries).

3. Internal Working

Cache hierarchy a query traverses, in order: browser cache → OS stub resolver cache → (maybe local router) → **recursive resolver** (ISP or 8.8.8.8/1.1.1.1) → root/TLD/authoritative as needed. Transport: UDP: 53 (≤~1232B with EDNS0), falls back to TCP on truncation (TC bit); modern clients may use DoT (853) or DoH (443).

4. Packet Flow Explanation

```
app getaddrinfo("api.example.com")
1. stub -> recursive: A api.example.com? (recursion desired)
   [recursive cache miss ->]
2. recursive -> root: who handles .com?      -> NS list for .com (cached ~days)
3. recursive -> .com TLD: who handles example.com? -> NS + glue (cached ~2d)
4. recursive -> ns1.example.com: A api.example.com?
   -> answer: api.example.com CNAME lb.example.com; lb A 203.0.113.7 TTL 60
5. recursive caches all, returns to stub; stub caches; app connects.
Steps 2-3 skipped ~always (cached); typical resolution: 1 round trip to
the recursive (1-30ms), worst case cold: 4+ round trips (100-400ms).
```

5. ASCII Diagram

```
app -> [browser cache] -> [OS cache] -> [recursive resolver]
                                     | (cache miss path)
                                     v
                                [root] -> ".com is over there"
                                [.com TLD] -> "example.com NS = ns1..."
                                [authoritative ns1.example.com] -> A record
Each arrow answered from cache when possible; TTL controls freshness.
```

6. Real Production Example

The 2021 **Facebook outage**: BGP withdrew routes to their authoritative DNS → all Facebook domains became unresolvable globally → even internal tooling (and door badges, reportedly) failed. Lesson interviewers expect: DNS is a dependency of *everything*, including your recovery path — keep out-of-band access.

7. Advantages

- Massive scale via caching + delegation (no central bottleneck); independent administration per zone; the indirection enables LB/failover/CDN routing (change the answer, not the clients).

8. Trade-offs

- Eventual consistency by design: changes propagate on TTL expiry, not push. “DNS propagation” = caches expiring at different times.
- Plaintext UDP by default (privacy → DoH/DoT); spoofable (→ 0x20 encoding, port randomization, DNSSEC).
- Adds a serial startup dependency + failure mode to every connection.

9. Common Mistakes

- Saying the stub/client contacts root servers (only recursives do).
- Believing DNS “pushes” updates — it’s pull + TTL expiry, period.
- Forgetting negative caching (NXDOMAIN is cached too, per SOA MINIMUM — a deploy-day trap: query a name *before* creating it and you cache its nonexistence).

10. Performance Impact

Cached: ~0–1ms. Recursive hit: 1–30ms. Full cold walk: 100–400ms — added to *first-connection* latency (why browsers do dns-prefetch). A slow/failing resolver stalls every new connection on the host: DNS timeouts are 5s-ish defaults — instant p99 catastrophe.

11. Common Interview Questions

1. Full walk of a cold DNS resolution — every actor.
2. Recursive vs iterative queries — who does which? (Stub → recursive is recursive; recursive → root/TLD/auth is iterative.)
3. Why is DNS over UDP, and when does it use TCP? (Truncation >1232B, zone transfers AXFR, DoT/DoH.)

12. Follow-up Questions

- “What’s EDNS0?” → extension mechanism: bigger UDP payloads, ECS (client-subnet), cookies.
- “What are glue records?” → A records for nameservers *inside* the zone they serve, provided by the parent to break the chicken-and-egg.
- “CNAME at zone apex — why illegal, what instead?” → CNAME can’t coexist with SOA/NS at apex; use ALIAS/ANAME/CNAME-flattening (Route53 alias, Cloudflare flattening).

13. Debugging Scenarios

- `dig +trace name` — replays the full delegation walk; `dig @resolver name` — tests a specific resolver; compare answers across resolvers for propagation issues.
- App resolves wrong/stale IP: check every cache layer (JVM caches forever by default with a security manager! `networkaddress.cache.ttl` — a notorious Java gotcha).

14. Best Practices

- Respect TTL strategy: lower TTL to 60s *before* a migration (a TTL in advance!), restore after.
- Run redundant NS across providers (post-Dyn-2016 lesson); monitor resolution latency + NXDOMAIN rates.
- In-app: honor TTLs, add jitter to re-resolution, never cache forever.

15. Practice Questions

1. You change an A record (TTL 300) at 12:00. A user still hits the old IP at 12:20. List every place the stale answer could live. (Recursive that cached at 11:59+300s edge... plus OS cache, browser cache, JVM cache, app pool holding old *connections* — the last one isn't even DNS!)
2. Why did queries for a brand-new subdomain fail for exactly 15 minutes after creation for some users? (Negative caching of NXDOMAIN from pre-creation queries; SOA MINIMUM=900.)

Topic 5.2 — Recursive Resolver

1. Why Interviewers Ask This

The recursive is where DNS's scale, performance, and security battles happen (cache poisoning, anycast, privacy). Also a system-design target: "design a public DNS resolver like 8.8.8.8."

2. Core Concept

The recursive resolver does the *work*: takes a stub's question, walks the hierarchy iteratively, validates, caches, answers. Public resolvers (Google 8.8.8.8, Cloudflare 1.1.1.1, Quad9) are globally **anycast**: one IP, hundreds of sites; BGP routes you to the nearest.

3. Internal Working

- Cache keyed by (name, type, class); honors TTLs; serves millions of QPS from memory.
- Poisoning defense: source-port randomization, query-ID randomness, 0x20 case randomization, DNSSEC validation (Kaminsky attack made this famous — worth naming).
- **Query coalescing**: 1000 concurrent misses for the same name → 1 upstream query (thundering-herd protection).
- **Serve-stale** (RFC 8767): answer expired records when authoritatives are unreachable — resilience feature that saved many during authoritative outages.
- **ECS (EDNS Client Subnet)**: recursive forwards client's /24 to authoritatives so CDNs can geo-route accurately despite centralized resolvers (privacy trade-off; 1.1.1.1 refuses it, 8.8.8.8 sends it — great nuance).

4. Packet Flow Explanation

```
10k clients ask "netflix.com" within 1s at one anycast site:
q1 arrives -> cache miss -> ONE iterative walk starts, others queue on it
walk: root(cached) -> .com(cached) -> ns.netflix.com -> A + TTL 60
all 10k get the answer; next 60s served from RAM at ~0 cost.
Cache eviction: LRU + TTL expiry; hot names essentially never leave.
```

5. ASCII Diagram

```

      stubs (millions)
      | UDP/DoH
+-----+
| recursive |   anycast 1.1.1.1 announced from 300+ sites
| [cache RAM] | BGP delivers you to the nearest
| coalescing | |----iterative----> root/TLD/authoritative
+-----+      (tiny fraction of queries)
poisoning defenses: rand port+ID+0x20, DNSSEC validate
```

6. Real Production Example

Cloudflare 1.1.1.1: anycast from 300+ cities, median latency <15ms globally. The 2016 **Dyn attack** hit *authoritatives* and took down Twitter/Spotify/GitHub for users whose recursives' caches expired — the incident behind “use multiple DNS providers.”

7. Advantages

- Centralizes caching + security for millions of clients; anycast gives locality + DDoS absorption; enterprise recursives add filtering/split-horizon.

8. Trade-offs

- Centralization: your DNS provider sees your entire browsing metadata (→ DoH debates); big-resolver outage = mass outage (8.8.8.8 or 1.1.1.1 blips make news).
- ECS trade-off: geo-accuracy vs privacy vs cache fragmentation (per-subnet cache entries).

9. Common Mistakes

- Conflating recursive and authoritative roles (“Route53 vs 8.8.8.8 do the same thing” — no: one *answers for zones*, the other *resolves for clients*).
- Ignoring that K8s clusters run their own recursive layer (CoreDNS) with its own failure modes (ndots: 5! — every lookup tries 4-5 suffixed variants first; a legendary latency gotcha).

10. Performance Impact

Resolver choice moves *every cold connection's* latency: nearby anycast 5–15ms vs overloaded ISP resolver 50–200ms. Cache hit ratios >95% at public resolvers; the tail (misses + retries) dominates perceived DNS slowness.

11. Common Interview Questions

1. Design a public DNS resolver — QPS, caching, anycast, poisoning defenses, coalescing.
2. How does anycast work and what breaks it? (BGP nearest-exit; per-flow instability is fine for UDP one-shots, tricky for TCP/DoH — mitigated by route stability + conn migration.)

3. What was the Kaminsky attack? (Race to forge responses using predictable query IDs + bailiwick trickery → fixed by entropy + DNSSEC.)

12. Follow-up Questions

- “Why does DoH centralize trust and why do enterprises hate it?” → bypasses local split-horizon/ filtering; browser picks resolver, not the network admin.
- “How does serve-stale change the failure story?” → authoritative outage degrades to ‘frozen answers’ instead of hard failure.

13. Debugging Scenarios

- Some users resolve to wrong region: their recursive is far away and no ECS → CDN geolocates the *resolver* not the user; enable ECS or use anycast for the service itself.
- K8s pod does 5 DNS lookups per external call (ndots) → set `ndots:1` /FQDN with trailing dot, or NodeLocal DNSCache.

14. Best Practices

- Applications: use the local caching layer (systemd-resolved/NodeLocal), set sane timeouts (<1s) + retries against 2 resolvers.
- Infra: NodeLocal DNSCache in K8s (contrack races on UDP DNS caused infamous 5s timeouts — the `use-vc` /race bug: two parallel A+AAAA queries hit a contrack race; mention = strong ops signal).

15. Practice Questions

1. Your service’s p99 shows a 5s plateau exactly, only in Kubernetes. Explain the classic cause. (Parallel A+AAAA UDP queries, contrack insert race drops one, glibc waits 5s timeout; fixes: NodeLocal cache, single-request-reopen, TCP DNS.)
2. Design DNS for an internal platform: split-horizon (internal zones), recursive layer with serve-stale, forwarding rules to corp + public, and why you still honor low TTLs internally.

Topic 5.3 — Authoritative Server

1. Why Interviewers Ask This

This is *your* side of DNS as a service owner — where records, zone design, failover, and traffic-steering live. System design rounds (“multi-region failover”) land here.

2. Core Concept

Authoritative servers hold the zone’s actual data and answer without recursion. Zones are delegated via NS records in the parent. Managed authoritatives (Route53, Cloudflare DNS, NS1) add health checks, latency/ geo/weighted routing policies — DNS becomes a **global load balancer (GSLB)**.

3. Internal Working

- Zone data: SOA (serial, refresh, MINIMUM/negative-TTL), NS, then records. Replication: primary → secondary via AXFR/IXFR triggered by NOTIFY, or provider-proprietary sync.

- Policy answers: the server computes the response per query using health state + client (resolver or ECS) location: latency-based, geo, weighted (canary!), failover records.
- Anycast fleets: the same NS IPs announced globally; capacity against DDoS.

4. Packet Flow Explanation

Health-checked failover (Route53-style):

```
- api.example.com: PRIMARY A 1.2.3.4 (health check HTTP /healthz)
                     SECONDARY A 5.6.7.8
t0: primary healthy -> all queries answered 1.2.3.4 (TTL 60)
t1: health checkers (multi-region, e.g. 3/5 must fail) mark primary DOWN
t2: authoritative flips answers to 5.6.7.8 IMMEDIATELY
t3: world converges as 60s caches expire => failover time ≈ detection + TTL
    (+ misbehaving caches that ignore TTLs – always mention the stragglers)
```

5. ASCII Diagram

```
      [.com parent: NS -> ns{1..4}.example-dns.net]
              |
anycast authoritative fleet (all 4 NS names, many sites)
zone: SOA | NS | A/AAAA | CNAME | MX | TXT
+ routing policies: geo  latency  weighted  failover
                   \      |      |      /
                   health checkers feed state in real time
```

6. Real Production Example

Route53's SLA is 100% (achieved via massive anycast + 4 NS in different TLDs). Weighted records at 1%/99% are a standard canary-release mechanism at companies without smarter L7 tooling. The Dyn 2016 attack → “dual authoritative providers” became a resilience checklist item (requires keeping zones in sync across providers).

7. Advantages

- Global steering with zero client changes; provider anycast absorbs DDoS; policies (geo/latency/weighted/failover) give you GSLB without deploying anything.

8. Trade-offs

- Failover speed bounded by TTL + detection (~seconds to minutes) — much slower than an LB's health check (sub-second). DNS steers *coarse* traffic; LBs handle *fast* failover.
- Some resolvers/apps ignore TTLs → long tails on any change.
- ECS-less resolvers reduce geo accuracy (you locate the resolver, not the user).

9. Common Mistakes

- Designing “instant failover via DNS” with TTL 1s — resolver floods + many caches clamp minimum TTLs; realistic floor ~30–60s.
- Single authoritative provider as a SPOF (Dyn lesson).
- Forgetting SOA MINIMUM controls *negative* caching — failing lookups for a new record stick around.

10. Performance Impact

Authoritative latency only affects cache misses, but *your* TTL choices set the caching economics: TTL 60 = agile + more misses; TTL 86400 = cheap + slow to change. Standard pattern: long TTLs on stable records (NS, MX), short (60–300s) on traffic-steering records.

11. Common Interview Questions

1. Design multi-region active-passive failover using DNS — walk the timeline of a region failure.
2. How do weighted/latency/geo policies work under the hood?
3. AXFR vs IXFR vs NOTIFY?

12. Follow-up Questions

- “Why 4+ NS records in different TLDs?” → survive TLD-level or provider-level failures.
- “How would you canary 1% of traffic with DNS, and what’s the gotcha?” → weighted records; gotcha = resolver caching quantizes the split per-resolver-population, not per-user (big ISPs’ resolvers = big chunks).

13. Debugging Scenarios

- `dig +norec @ns1.example.com name` — query the authoritative directly, bypassing caches; compare across your NS fleet for sync issues (check SOA serial on each).
- Region failover “didn’t happen” for one ISP → their resolver caches beyond TTL; measure real-world convergence with distributed probes (RIPE Atlas).

14. Best Practices

- Health-checked failover records + TTL 60 on steering names; long TTLs elsewhere.
- Two providers or provider + self-hosted secondaries; automate zone sync; monitor SOA serial drift.
- Pre-lower TTLs before planned migrations (one full old-TTL in advance).

15. Practice Questions

1. Timeline: primary region dies at $t=0$; health check = 3 failures $\times$ 10s interval; TTL=60. When do 50%/95% of users converge? ($\sim t=30$ s flip + cache expiry distribution $\approx$ 50% by ~ 60 s, 95% by ~ 90 – 120 s, stragglers beyond.)
2. Your zone’s two providers drifted (one has old records) — what monitoring catches this? (Cross-provider SOA serial + record diffs from external probes.)

Topic 5.4 — DNS Caching & TTL

1. Why Interviewers Ask This

TTL trade-offs are pure engineering judgment — agility vs load vs blast radius — and caching bugs cause real outages (stale endpoints, negative-cache surprises, JVM-forever caches).

- Low TTLs: more resolver load, more cold-lookup latency, higher exposure to DNS outages (cache runs dry quickly).
- Layered caches multiply debugging surface (“which of 5 caches is stale?”).

9. Common Mistakes

- Assuming a TTL change takes effect immediately — the *old* TTL governs how long old answers (with the old TTL) persist. Lower TTL one full old-TTL *before* the change.
- Ignoring negative caching during launches.
- Treating DNS convergence as a switch instead of a decay curve.

10. Performance Impact

Cache hit: <1ms. Miss: 10–400ms. For high-QPS microservices doing per-request resolution with low TTL, DNS itself becomes a top-3 dependency: measure lookup rate; NodeLocal/daemon caches cut 99% of it.

11. Common Interview Questions

1. Walk every cache between a browser and your record, with realistic TTL behavior at each.
2. How do you choose TTLs? (Steering/failover names: 60s; stable infra: hours-day; apex NS: days.)
3. Why didn't lowering TTL to 30 make your migration instant?

12. Follow-up Questions

- “How do connection pools interact with DNS failover?” → they don't re-resolve until connections break — failover must actively kill connections or use TTL-aware pools.
- “What clamps TTLs in the wild?” → resolvers with min/max TTL policies (min 30–60s common, max 1–7d), serve-stale extensions.

13. Debugging Scenarios

- `dig name` (remaining TTL from recursive) vs `dig @ns1.provider name` (fresh TTL from authoritative) — instantly localizes staleness.
- One host resolves differently from others: check `/etc/hosts` (!), local daemon cache (`resolvectl flush-caches`), and app-runtime cache before blaming DNS.

14. Best Practices

- TTL playbook: pre-lower before changes; steering records 60s; verify convergence with external probes, not just your laptop.
- Kill JVM-forever caching explicitly; set nginx `resolver` with `valid=`; make clients re-resolve on connection failure with jitter.

15. Practice Questions

1. Record has TTL 86400. You need to migrate in 2 hours. What's your plan and its risk? (Lower TTL now → old 86400 answers persist up to 24h regardless → either wait, or plan dual-serving both IPs during overlap — the real answer is “serve on both endpoints during transition.”)

2. After failover, 5% of traffic still hits the dead IP after 24h. List three non-TTL causes. (Hardcoded IPs/hosts files, forever-caching runtimes, pooled connections/keepalive to a zombie host behind a still-up TCP endpoint.)

Topic 5.5 — CDN Integration (DNS-Based Traffic Steering)

1. Why Interviewers Ask This

This ties DNS into system design's favorite component (CDN, Module 7): *how does a user in Tokyo reach the Tokyo edge?* The answer is DNS (or anycast, or both) — and interviewers expect the comparison.

2. Core Concept

Two steering mechanisms: 1. **DNS-based**: CDN's authoritative answers differently per querying resolver's location (+ECS) → returns nearby edge IPs. (Akamai's classic approach; TTL ~20–60s.) 2. **Anycast**: one IP announced everywhere; BGP routes to nearest site (Cloudflare's approach). DNS then plays a smaller role (same answer globally). Most real CDNs blend both.

3. Internal Working

DNS-based path: `www.shop.com` → CNAME `shop.cdn-provider.net` → CDN's authoritative applies geo/latency/load maps (built from real-time measurements) → returns edge VIPs for that region. Accuracy depends on resolver location (hence ECS). Anycast path: routing table does the work; steering granularity = BGP, not per-user; site drain = withdraw announcement.

4. Packet Flow Explanation

```
Tokyo user, DNS-steered CDN:
1. stub -> local recursive (Tokyo ISP)
2. recursive -> cdn authoritative: A shop.cdn.net? (+ECS 203.0.113.0/24)
3. authoritative: "that /24 maps to region ap-northeast; edge pool T"
   -> A 151.101.x.x TTL 30
4. user connects to Tokyo edge; TLS+content served ~5ms away.
Failover: edge pool T unhealthy -> authoritative answers Osaka pool
within seconds (TTL 30 bounds client convergence).
```

5. ASCII Diagram

DNS steering: user->resolver->[CDN auth DNS] "you look Japanese, here's Tokyo edge IP" + fine control, per-resolver - resolver-location errors - TTL-bounded failover	Anycast: user ----> 104.16.x.x BGP: nearest of 300 sites wins automatically + instant failover (routing) - no per-user control - flow instability edge cases (TCP)
--	---

6. Real Production Example

- **Akamai**: DNS-based (deep in-ISP deployments; CNAME chains through `akadns.net` with 20s TTLs).
- **Cloudflare/Google/Fastly**: anycast-first (fewer, larger sites; instant BGP failover).
- **Netflix Open Connect**: steering decided by their control plane, then the *client app* is handed specific OCA URLs — a third pattern: application-level steering (most precise; requires owning the client).

7. Advantages

DNS steering: fine-grained (per-resolver/per-ECS-prefix), policy-rich (load, cost), no BGP expertise required. Anycast: zero-TTL failover, absorbs DDoS across all sites, no resolver-accuracy problem.

8. Trade-offs

DNS steering: wrong-resolver mislocation (VPN/8.8.8.8 users without ECS), convergence bounded by TTL, resolver TTL-clamping. Anycast: BGP route changes can mid-stream break TCP (rare; QUIC's conn-ID migration helps — nice cross-module tie), per-user steering impossible, capacity engineering per-site is harder.

9. Common Mistakes

- “CDN = anycast” or “CDN = DNS tricks” as if exclusive — real answer: both, layered.
- Forgetting the resolver-location problem (the user's location is *inferred* from their resolver absent ECS).
- Proposing DNS steering for sub-second failover (TTL floor makes it seconds-minutes).

10. Performance Impact

Steering accuracy is worth 10–100ms per connection (nearest vs wrong-continent edge). Bad steering shows up as a *bimodal* latency distribution per region — the diagnostic signature to mention.

11. Common Interview Questions

1. How does a user get routed to the nearest CDN edge? (Expect both mechanisms + hybrid.)
2. What breaks user-to-edge mapping? (Public resolvers w/o ECS, VPNs, mobile CGNAT egress far away.)
3. How does a CDN drain a failing PoP under each scheme? (DNS: stop answering it, wait TTL; anycast: withdraw BGP, instant.)

12. Follow-up Questions

- “Design the measurement system behind the geo map.” → real-user measurements (RUM beacons), probe meshes, per-prefix latency tables refreshed continuously.
- “Why do CDN DNS names have such short TTLs, and what load does that create?” → agility for steering/failover; absorbed because the CDN *is* the authoritative at massive scale.

13. Debugging Scenarios

- Users in India served from Europe: `dig` from their network vs `dig +subnet=<their /24>` against CDN auth — compare answers; check if their ISP resolver egresses elsewhere or strips ECS.
- After PoP outage, one ISP's users stuck on dead edge for 30 min → resolver ignoring 30s TTL; escalate to CDN (they often also keep dead-VIP anycast fallbacks).

14. Best Practices

- Use CNAME to CDN-managed names (let them own steering + TTLs); don't pin edge IPs anywhere.
- For your own multi-region APIs: latency-based DNS + per-region LBs + anycast only if you have BGP capability; measure with RUM, not just synthetic probes.

15. Practice Questions

1. Design global routing for `api.yourco.com` across 3 regions: choose DNS policy (latency-based, health-checked, TTL 60), explain failover timeline, and what anycast would add/cost.
2. A customer on 8.8.8.8 in Sydney gets US edges. Explain the mechanism and two fixes. (No/stripped ECS → resolver geolocated to US egress... actually 8.8.8.8 sends ECS and is anycast-local; better example: corporate VPN egress in US → fixes: ECS-aware steering can't help VPN egress; anycast service IPs or client-side region selection.)

MODULE 5 — One-Page Cheat Sheet

RESOLUTION	stub -> recursive -> (root -> TLD -> authoritative), all cached recursive does the walk; stub never talks to root UDP:53 (<=1232B EDNS) -> TCP on TC; DoT:853 DoH:443
RECORDS	A/AAAA CNAME (never at apex; use ALIAS/flattening) NS MX TXT SRV SOA(serial, MINIMUM = negative TTL)
CACHES	browser -> OS -> daemon -> recursive [+ JVM! nginx-startup! connection pools pin IPs – not DNS but acts like it]
TTL LAW	change visible only as caches expire; pre-lower TTL one full old-TTL before migration; NXDOMAIN cached too (SOA MINIMUM)
FAILOVER	health-checked records + TTL60 => ~1-2 min convergence + stragglers DNS = coarse/slow steering; LB = fast failover. Use both.
STEERING	DNS-based (per-resolver geo, ECS) vs anycast (BGP, instant) real CDNs: hybrid. Resolver mislocation = classic bug.
SECURITY	spoofing -> rand port/ID/0x20, DNSSEC; reflection -> RRL, cookies
K8S TRAPS	ndots:5 (5 lookups per name) conntrack race -> 5s timeouts -> NodeLocal DNSCache, FQDN with trailing dot
TOOLS	dig +trace dig @ns dig +nored resolvectl RIPE Atlas
NUMBERS	cache hit <1ms recursive 1-30ms cold walk 100-400ms typical steering TTL 20-60s resolver TTL clamps exist

MODULE 5 — Top Interview Questions

1. Cold resolution walk, every actor and cache. (The classic.)
2. Recursive vs authoritative — roles, owners, failure modes (Dyn vs Facebook outages as case studies).
3. Design DNS-based multi-region failover; give the honest convergence timeline.
4. Why did your DNS change take 24h to fully propagate despite TTL 300?
5. How do CDNs route users to the nearest edge — DNS vs anycast trade-offs?
6. Explain DNS cache poisoning and modern defenses.
7. The Kubernetes 5s DNS timeout — mechanism and fixes.
8. Weighted DNS canary: how, and why the split is approximate.

MODULE 5 — Common Mistakes

- Stub-talks-to-root misconception; push-vs-pull confusion (“propagation”).

- Ignoring negative caching, TTL clamping, JVM/nginx/pool stragglers.
- Designing sub-second failover on DNS; single authoritative provider.
- Forgetting DNS is in the critical path of every cold connection — and its 5s default timeouts destroy p99s.
- CNAME at apex; querying a name before it exists during launches.

MODULE 5 — Mock Interview (12 min)

Q1. “Your primary region just died. You’re on DNS failover. Narrate the next 5 minutes for users, honestly.”

Strong answer: 0–30s: health checks confirm (N-of-M probes) and flip answers; 30–90s: TTL-expiry decay curve — traffic drains as caches expire; minutes+: stragglers (clamping resolvers, JVM caches, pooled connections into dead sockets — those hang on TCP retransmits ~15 min unless RST’d) → real designs pair DNS with in-region LB failover and active connection termination; state you’d measure the decay with external probes.

Q2. “A user reports your site resolves to different IPs on phone vs laptop on the same WiFi. Bug?” *Strong*

answer: usually not: different caches snapshotting a rotating/steered record at different times, or phone using DoH (bypasses router DNS) getting different steering; would compare `dig` outputs, TTL remainders, resolver identity (`resolvetl` /whoami.akamai queries); only investigate as an incident if one IP is unhealthy.

Q3. “Design DNS for a SaaS with 50k customer vanity domains.” *Strong answer:* customers CNAME to

`customers.saas.com` (or apex via ALIAS/flattening); our side: managed authoritative with health-checked steering, automated ACME issuance keyed on DNS validation, per-domain cert store at edge (SNI), monitoring for broken customer delegations; discuss the apex problem and provider-redundancy explicitly.